

GRAIL: A Deep-Granularity Hybrid Resonance Framework for Real-Time Agent Discovery via SLM-Enhanced Indexing

Jinliang Xu*

China Academy of Information and Communications Technology, Beijing, China
xujinliang@caict.ac.cn

Abstract—As the ecosystem of Large Language Model (LLM)-based agents expands rapidly, efficient and accurate Agent Discovery becomes a critical bottleneck for large-scale multi-agent collaboration. Existing approaches typically face a dichotomy: either relying on heavy-weight LLMs for intent parsing, leading to prohibitive latency (often exceeding 30 seconds), or using monolithic vector retrieval that sacrifices semantic precision for speed. To bridge this gap, we propose GRAIL (Granular Resonance-based Agent/AI Link), a novel framework achieving sub-400ms discovery latency without compromising accuracy. GRAIL introduces three key innovations: (1) SLM-Enhanced Prediction, replacing the generalized LLM parser with a specialized, fine-tuned Small Language Model (SLM) for millisecond-level capability tag prediction; (2) Pseudo-Document Expansion, augmenting agent descriptions with synthetic queries to enhance semantic density for robust dense retrieval; and (3) MaxSim Resonance, a fine-grained matching mechanism computing maximum similarity between user queries and discrete agent usage examples, effectively mitigating semantic dilution. Validated on AgentTaxo-9K, our new large-scale dataset of 9,240 agents, GRAIL reduces end-to-end discovery latency by over $79\times$ compared to LLM-parsing baselines, while significantly outperforming traditional vector search in Recall@10. This framework offers a scalable, industrial-grade solution for the real-time “Internet of Agents.”

Index Terms—Multi-Agent Systems, Agent Discovery, Small Language Models, Hybrid Retrieval, Neural Search.

I. INTRODUCTION

The rapid evolution of Large Language Models (LLMs) has catalyzed a paradigm shift from isolated conversational bots to interconnected Multi-Agent Systems (MAS) [1], [2]. As envisioned in the concept of the “Internet of Agents”, thousands of specialized agents—ranging from coding assistants to financial analysts [3], [4]—are being deployed to collaboratively solve complex tasks. In this ecosystem, *Agent Discovery*, the process of accurately matching a user’s natural language intent to the most capable agent among a massive candidate pool, has emerged as a critical infrastructure challenge [5], [6].

Despite the proliferation of agent protocols [7], achieving real-time, high-precision agent discovery remains an unsolved problem. Existing approaches predominantly fall into two categories, each with significant limitations:

1. LLM-Centric Parsing (High Latency): Some frameworks rely on general-purpose LLMs (e.g., Gemini 3, DeepSeek) to parse user intent and generate structured queries (e.g., SQL or filtered JSON) before searching the database.

While highly accurate in understanding complex intent, this “Think-then-Lookup” paradigm incurs prohibitive latency [8]. Our empirical analysis shows that the LLM parsing stage alone often consumes over 30 seconds per query, rendering it unsuitable for real-time interaction or high-frequency agent-to-agent negotiation.

2. Monolithic Dense Retrieval (Low Precision): Alternatively, lightweight approaches utilize vector databases to perform dense retrieval on agent descriptions. While efficient (achieving millisecond-level response), these methods typically compress an agent’s diverse capabilities (descriptions, tags, and usage examples) into a single embedding vector. This “coarse-grained” compression leads to semantic drift, where subtle user constraints (e.g., specific API version or pricing model) are lost, resulting in the retrieval of irrelevant agents.

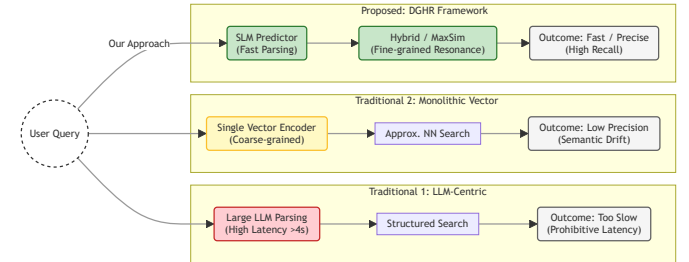


Fig. 1: Comparison of Agent Discovery Paradigms. *LLM-Centric Parsing* suffers from high latency ($> 4s$) due to heavy inference. *Monolithic Vector Retrieval* sacrifices semantic precision for speed, leading to low recall. Our proposed **GRAIL Framework** leverages a specialized SLM predictor and fine-grained MaxSim resonance to achieve sub-400ms latency without compromising accuracy.

To bridge this gap between the precision of LLM reasoning and the speed of vector search, we propose **GRAIL** (Granular Resonance-based Agent/AI Link), a framework designed for real-time (sub-400ms) discovery of agents at scale. As shown in Fig. 1, unlike previous methods that treat agent metadata as a monolithic text block, GRAIL decouples the discovery process into three granular dimensions: *Tags*, *Context*, and *Intent*. Instead of relying on heavy-weight LLMs for online parsing, GRAIL employs a fine-tuned Small Language Model (SLM, 3B parameters) to predict capability tags in mil-

liseconds, enabling rapid candidate filtering. Furthermore, we introduce a *MaxSim Resonance* mechanism that computes the interaction between the user query and discrete agent usage examples (shots), ensuring precise intent matching without the computational cost of full-text cross-encoders.

- **We propose GRAIL, a SLM-driven, deep-granularity framework for large-scale agent discovery that targets the latency–precision trade-off.** By replacing heavy-weight LLM parsing with a compact SLM and a streamlined retrieval pipeline, GRAIL achieves sub-400ms end-to-end latency while improving top- K accuracy over LLM-centric routing and standard dense retrieval.
- **We design a Deep-Granularity Hybrid Indexing scheme that separates agent signals into Tag, Context, and Intent dimensions.** By combining SLM-predicted sparse tags with Pseudo-Document–expanded dense descriptions, it enables robust hybrid recall and reduces vocabulary mismatch between metadata and user queries.
- **We develop MaxSim Resonance, a fine-grained matching mechanism over agent usage-example matrices instead of monolithic embeddings.** Late-interaction style maximum similarity between queries and discrete capability modes mitigates semantic dilution in multi-functional agents and yields clear gains in Recall@ K and MRR.
- **We release AgentTaxo-9K, a hierarchically structured benchmark for agent discovery with 9,240 agents and 27,720 queries, and use it to evaluate GRAIL.** Based on an industry–subdomain–function taxonomy and diverse query intents, AgentTaxo-9K provides a standardized testbed where GRAIL consistently outperforms strong baselines in effectiveness and efficiency.

II. RELATED WORK

Our work sits at the intersection of agent interconnection protocols, efficient neural routing, and fine-grained information retrieval. In this section, we review the existing landscape and position GRAIL as a complementary indexing layer for the emerging “Internet of Agents” [9].

A. Agent Interconnection Protocols & Discovery Standards

The infrastructure for the Internet of Agents is being shaped by a series of emerging communication protocols that standardize agent identity and capability description [10], [11].

1. Agent Card-Based Protocols (e.g., Google A2A): Recent industry initiatives, such as the **Google A2A (Agent-to-Agent)** protocol, utilize *Agent Cards* as standardized manifests to encapsulate agent identities, capabilities, and endpoints [12], [13]. While Agent Cards provide a unified metadata schema for interoperability, existing implementations often rely on static registries or direct handshakes, lacking a high-performance engine for fuzzy semantic discovery [14].

2. Model Context Protocol (MCP): Anthropic’s **Model Context Protocol (MCP)** standardizes the connection between LLMs and external tools, functioning as a “USB-C for AI” [15]. However, MCP primarily addresses tool binding within

a host’s context window. As the number of tools grows, static context injection becomes infeasible, necessitating active discovery mechanisms like “MCP-Zero” [16].

3. Network-Layer Discovery (e.g., ANP): The **Agent Network Protocol (ANP)** and similar directory services (e.g., Cisco ADS) address decentralized agent reachability using Distributed Hash Tables (DHTs) or DNS-SD variants [17], [18]. These network-centric approaches excel at routing by ID but struggle with complex, natural language intent matching. GRAIL complements these protocols by serving as a dynamic, semantic indexing engine that operates on top of standardized metadata (e.g., Agent Cards or MCP manifests).

B. Efficient Routing via Small Language Models

The paradigm of “LLM-as-a-Router” has evolved from using giant models to specialized Small Language Models (SLMs). 1) **LLM Tool Learning:** Pioneering works such as **Gorilla** [19] and **ToolLLM** [20] have demonstrated that Large Language Models can be fine-tuned to master thousands of APIs. However, relying on general-purpose LLMs (e.g., Gemini 3) for intent routing introduces significant latency (often > 30 s), creating a bottleneck for real-time agent interaction [21], [22]. 2) **SLM and Knowledge Distillation:** To address the latency challenge, recent research advocates for the use of SLMs (parameters < 1 B) as efficient neural routers [22]. Through *Knowledge Distillation*, capabilities from larger teacher models are transferred to SLMs, enabling them to perform specific intent classification tasks with comparable accuracy but at a fraction of the inference cost (e.g., < 400 ms) [21]. GRAIL adopts this strategy, utilizing a fine-tuned SLM for rapid tag prediction to filter candidates before expensive retrieval operations.

C. Neural Retrieval & Fine-Grained Late Interaction

Retrieving the correct agent from a massive pool requires overcoming the “semantic dilution” problem inherent in traditional dense retrieval. **1. Limitations of Monolithic Embeddings:** Standard bi-encoder architectures (e.g., DPR) compress an entire document into a single vector [23], [24]. In the context of multi-functional agents, this compression leads to *context dilution*, where specific capabilities are averaged out, making precise matching difficult [25]. **2. Late Interaction (MaxSim):** To preserve fine-grained semantic signals, **ColBERT** introduced the concept of *Late Interaction*, which computes the maximum similarity (MaxSim) between query tokens and document tokens [26]. Recent optimizations like **PLAID** have proven the viability of this approach in low-latency settings [27]. GRAIL adapts the MaxSim operator from the token level to the “example level,” computing resonance between user queries and discrete agent usage examples. **3. Pseudo-Document Expansion (PDE):** To bridge the vocabulary mismatch between user queries and technical agent descriptions, generative query expansion methods (e.g., **Doc2Query**) have been widely adopted [28]. GRAIL integrates PDE to augment agent metadata, enhancing the semantic density of the context index.

III. THE GRAIL FRAMEWORK

In this section, we present the **GRAIL** framework. We first formulate the agent discovery problem, then detail the offline construction of our tri-dimensional index, and finally describe the online SLM-driven retrieval and MaxSim re-ranking process. Fig. 2 illustrates the overall architecture. The system operates in two distinct phases: **(1) Offline Deep-Granularity Indexing**: Agent metadata is processed into three independent indices—Sparse (Tags), Context (Pseudo-Doc Augmented Vectors), and Intent (Fine-Grained Example Matrices). **(2) Online SLM-Driven Discovery**: A lightweight SLM predictor (3B params) accelerates intent parsing (< 50ms), driving a hybrid recall mechanism followed by the MaxSim Resonance re-ranking algorithm to ensure precise intent matching.

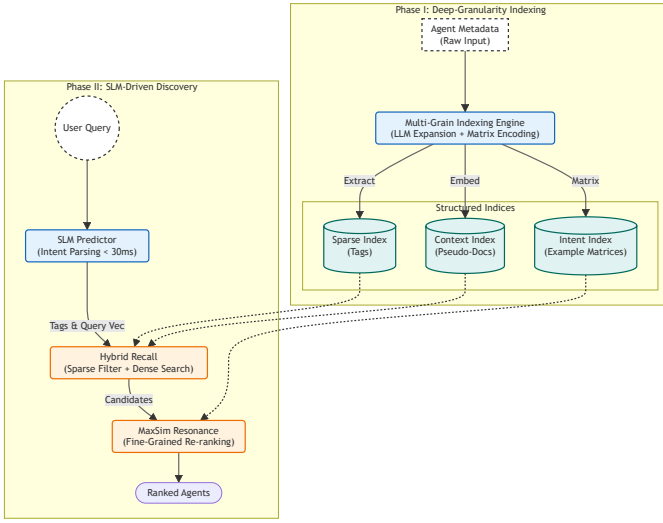


Fig. 2: The Overall Architecture of the GRAIL Framework.

A. Problem Formulation

To illustrate the structure of agents within our framework, consider the following example of an agent’s metadata (including *Description*, *Tags* and *Examples*), represented in JSON format:

```
{
  "Name": "HR-Core Automator",
  "Description": "This agent is an expert in optimizing Human Resources workflows within Human Capital Management systems like ...",
  "Tags": ["Security", "Technology", "HR", "HCM", "Workday", ...],
  "Examples": [
    "Draft an API call to Workday to init the onboarding workflow ...",
    "Analyze the attached employee ..."
  ]
}
```

Let $\mathcal{A} = \{a_1, a_2, \dots, a_N\}$ denote a repository of N agents. Each agent a_i is defined by a tuple of structured metadata: $a_i = \langle \mathcal{T}_i, \mathcal{D}_i, \mathcal{E}_i \rangle$, where:

- $\mathcal{T}_i$ is a set of discrete capability tags (*Tags*), encompassing core domain areas and primary capabilities, alongside both high-frequency and low-frequency yet representative unique tags specific to the agent’s functionalities.
- $\mathcal{D}_i$ is a natural language description of the agent’s functionality (*Description*).
- $\mathcal{E}_i = \{e_{i,1}, e_{i,2}, \dots, e_{i,m}\}$ is a list of specific usage examples or few-shot prompts (*Examples*).

Given a user query q , the objective is to retrieve a ranked subset of agents $\mathcal{R} \subset \mathcal{A}$ ($|\mathcal{R}| \ll N$) such that the relevance score $S(q, a_i)$ is maximized for $a_i \in \mathcal{R}$, subject to a latency constraint $L < 400$ ms.

B. Offline: Deep-Granularity Indexing

Unlike monolithic retrieval systems that compress all agent information into a single vector, GRAIL constructs three distinct indices to capture capability at different granularities.

1) *Sparse Index via Explicit Tags (I_{tag})*: We construct an inverted index mapping each tag $t \in \bigcup \mathcal{T}_i$ to the list of agents possessing it. This serves as a hard constraint filter. To handle non-explicitly present synonymous tags, we employ a synonym expansion dictionary during index construction.

2) *Context Index via Pseudo-Document Expansion (I_{ctx})*: Standard dense retrieval on short descriptions ($\mathcal{D}_i$) often suffers from the “vocabulary mismatch” problem [29]. We adopt a document expansion strategy. For each agent a_i , we utilize a Large Language Model (LLM) to generate a set of potential user queries $\mathcal{Q}_i^{syn}$ based on $\mathcal{D}_i$ [30]. A pseudo-document $\mathcal{D}'_i$ is formed by concatenating the original description and the synthetic queries:

$$\mathcal{D}'_i = \mathcal{D}_i \oplus \text{Concat}(\mathcal{Q}_i^{syn}) \quad (1)$$

We then encode $\mathcal{D}'_i$ into a dense vector $\mathbf{v}_{ctx}^{(i)} \in \mathbb{R}^d$. This index I_{ctx} captures the broad semantic context of the agent.

It is important to note that synthetic queries serve a distinct purpose from the agent’s explicit usage examples for several reasons: 1) **Broader Semantic Coverage**: Synthetic queries, generated by an LLM based on the agent’s full description, can simulate a wider and more diverse range of user expressions, thus enhancing the broad semantic footprint of the agent for initial dense retrieval. 2) **Addressing Vocabulary Mismatch**: They are specifically designed to bridge the lexical gap between formal agent descriptions and varied natural language user queries, optimizing for general recall. 3) **Complementary Role**: The usage examples are often limited in number and represent specific, fine-grained interaction patterns. Their role in our framework is to preserve these distinct capability modes for subsequent detailed matching, which is fundamentally different from the broad contextual matching aim of this index. This strategic separation ensures that each index dimension is optimized for its unique contribution to the multi-stage discovery process.

3) *Intent Index via Fine-Grained Examples (I_{int})*: This is the core of our deep-granularity approach. Instead of averaging the embeddings of usage examples $\mathcal{E}_i$, which dilutes specific intent signals, we maintain the independence of each example. For each agent a_i , we encode its examples into a matrix of embeddings $\mathbf{M}_i \in \mathbb{R}^{m \times d}$:

$$\mathbf{M}_i = [\text{Enc}(e_{i,1}), \text{Enc}(e_{i,2}), \dots, \text{Enc}(e_{i,m})] \quad (2)$$

This matrix structure allows the system to preserve distinct capability modes (e.g., an agent capable of both “coding” and “plotting” will retain strong signals for both, rather than a blurred average).

Notably, the Sparse Index (I_{tag}) is a simple inverted index and does not require a vector database. In contrast, the construction of the Context Index (I_{ctx}) and Intent Index (I_{int}) necessitates the use of a vector database. For this, we leverage a pre-trained sentence transformer like BGE-M3 as the embedding model, which is used directly without further training. The resulting high-dimensional vectors are then organized into the required structures: I_{ctx} stores one vector per agent, representing its overall context, while I_{int} stores a matrix of vectors per agent, where each row corresponds to an encoded usage example. These structured vector data are then persistently stored in specialized vector databases or indexing libraries, enabling subsequent efficient retrieval and computation operations.

C. Online: SLM-Driven Hybrid Retrieval

To eliminate the latency bottleneck of LLM-based parsing, we introduce a pipelined retrieval mechanism driven by a Small Language Model (SLM).

1) *SLM Tag Prediction*: Upon receiving a query q , we employ a fine-tuned SLM to predict the most likely capability tags $\hat{\mathcal{T}}_q$. The SLM is trained on pairs of (synthetic query, tags) to minimize the cross-entropy loss. Through this training, the SLM learns to map diverse user intents to relevant tags, regardless of their granularity—encompassing both broad, high-level categories and specific, fine-grained unique tags that reflect specialized agent capabilities.

$$\hat{\mathcal{T}}_q = \text{SLM}_\theta(q) \quad (3)$$

Since the output vocabulary is restricted to the system’s tag set, inference time is strictly bounded.

2) *Hybrid Recall*: We perform parallel retrieval to generate a candidate set $\mathcal{C}$:

- **Sparse Path**: Retrieve agents whose capability tag set $\mathcal{T}_i$ contains *any* of the predicted tags in $\hat{\mathcal{T}}_q$. This effectively acts as a broad filter using an OR logic across the predicted tags. Let this set be $\mathcal{C}_{sparse}$.
- **Dense Path**: Encode q into $\mathbf{v}_q$ and perform Approximate Nearest Neighbor (ANN) search on I_{ctx} to retrieve top- k agents. Let this set be $\mathcal{C}_{dense}$.

The initial candidate set is the union: $\mathcal{C} = \mathcal{C}_{sparse} \cup \mathcal{C}_{dense}$. This hybrid approach ensures high recall by combining explicit intent (tags) with implicit semantic matching (vectors).

Algorithm 1: GRAIL Online Discovery Process

Require: Query q , Indices $I_{tag}, I_{ctx}, I_{int}$, SLM model $\mathcal{M}$
Ensure: Top- K ranked agents $\mathcal{R}$

- 1: **Step 1: SLM Prediction**
 - 2: $\hat{\mathcal{T}}_q \leftarrow \mathcal{M}.\text{predict}(q)$
 - 3: $\mathbf{v}_q \leftarrow \text{Encoder}(q)$
 - 4: **Step 2: Hybrid Recall**
 - 5: $\mathcal{C}_{sparse} \leftarrow I_{tag}.\text{lookup}(\hat{\mathcal{T}}_q)$
 - 6: $\mathcal{C}_{dense} \leftarrow I_{ctx}.\text{search}(\mathbf{v}_q, \text{top_k} = 50)$
 - 7: $\mathcal{C} \leftarrow \mathcal{C}_{sparse} \cup \mathcal{C}_{dense}$
 - 8: **Step 3: MaxSim Re-ranking**
 - 9: **for** each agent a_i in $\mathcal{C}$ **do**
 - 10: Load example matrix $\mathbf{M}_i$ from I_{int}
 - 11: $S_{res} \leftarrow \max(\mathbf{v}_q \cdot \mathbf{M}_i)$
 - 12: $S_{final} \leftarrow \alpha(\mathbf{v}_q \cdot \mathbf{v}_{ctx}^{(i)}) + (1 - \alpha)S_{res}$
 - 13: **end for**
 - 14: Sort $\mathcal{C}$ by S_{final} descending
 - 15: **return** Top- K agents from $\mathcal{C}$
-

D. MaxSim Resonance Re-ranking

The candidate set $\mathcal{C}$ is typically small (e.g., $|\mathcal{C}| = 50$). We perform a fine-grained re-ranking using the *MaxSim Resonance* score. For each candidate agent $a_i \in \mathcal{C}$, we compute the interaction between the query vector $\mathbf{v}_q$ and the agent’s example matrix $\mathbf{M}_i$. The resonance score is defined as the maximum cosine similarity found across all examples:

$$S_{res}(q, a_i) = \max_{j=1}^m \left(\frac{\mathbf{v}_q \cdot \mathbf{e}_{i,j}}{\|\mathbf{v}_q\| \|\mathbf{e}_{i,j}\|} \right) \quad (4)$$

Interpretation: This operation effectively measures if the user’s query “resonates” with *any single specific capability* demonstrated in the agent’s examples. Unlike average pooling, MaxSim is robust to noise; even if 9 out of 10 examples are irrelevant, a single high-match example triggers a high relevance score.

The final ranking score S_{final} is a weighted sum of the context score and the resonance score:

$$S_{final}(q, a_i) = \alpha \cdot (\mathbf{v}_q \cdot \mathbf{v}_{ctx}^{(i)}) + (1 - \alpha) \cdot S_{res}(q, a_i) \quad (5)$$

where α is a hyperparameter balancing broad context context and specific intent. The detailed pseudocode for this entire online discovery pipeline is provided in Algorithm 1.

The workflow of the GRAIL online discovery process is as shown in Fig. 3. The Discovery Engine coordinates the intent parsing (SLM/BGE), performs parallel hybrid recall across sparse (I_{tag}) and dense (I_{ctx}) indices, and executes MaxSim resonance re-ranking using fine-grained example matrices (I_{int}).

E. Theoretical Analysis: The Signal Dilution Problem

To rigorously justify the necessity of the MaxSim Resonance mechanism, we analyze the behavior of relevance scores under the *multi-skilled agent assumption*.

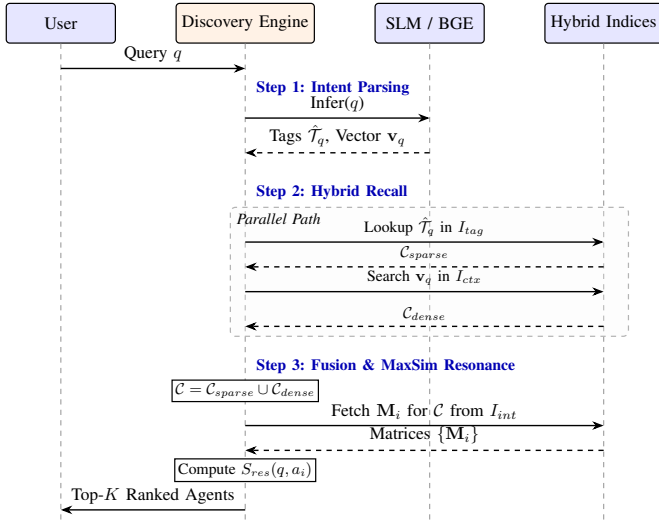


Fig. 3: Workflow of the GRAIL Online Discovery Process.

Theorem 1 (Semantic Dilution in Mean Pooling). *Let an agent a possess m distinct capabilities represented by normalized embedding vectors $\mathcal{E} = \{\mathbf{e}_1, \dots, \mathbf{e}_m\}$. Let q be a user query perfectly aligned with a specific capability $\mathbf{e}_k$ (i.e., $q \cdot \mathbf{e}_k = 1$). If the agent’s capabilities are distinct (orthogonal in high-dimensional space), the relevance score retrieved by monolithic mean-pooling decays linearly with m , whereas MaxSim preserves the unitary signal strength.*

Proof: Let $\mathbf{v}_{avg}$ be the monolithic vector representation of agent a used in traditional dense retrieval, typically computed via mean pooling:

$$\mathbf{v}_{avg} = \frac{1}{m} \sum_{i=1}^m \mathbf{e}_i \quad (6)$$

Assume the embedding space is high-dimensional (e.g., $d = 768$) such that distinct capabilities are approximately orthogonal. That is, for $i \neq j$, $\mathbf{e}_i \cdot \mathbf{e}_j \approx \epsilon$, where $\epsilon \rightarrow 0$.

Case 1: Traditional Mean Pooling Score (S_{avg}). The relevance score for query $q = \mathbf{e}_k$ is:

$$S_{avg} = q \cdot \mathbf{v}_{avg} = \mathbf{e}_k \cdot \left(\frac{1}{m} \sum_{i=1}^m \mathbf{e}_i \right) \quad (7)$$

$$= \frac{1}{m} \left(\mathbf{e}_k \cdot \mathbf{e}_k + \sum_{i \neq k} \mathbf{e}_k \cdot \mathbf{e}_i \right) \quad (8)$$

$$\approx \frac{1}{m} (1 + (m-1)\epsilon) \approx \frac{1}{m} \quad (9)$$

As the number of capabilities m increases (e.g., a “Swiss Army Knife” agent), the retrieval score S_{avg} asymptotically approaches 0. This phenomenon, which we term *Semantic Dilution*, causes capable agents to be buried in the ranking list by specialized but irrelevant agents.

Case 2: GRAIL MaxSim Score (S_{max}). In our framework, the relevance score is determined by the resonance mechanism:

$$S_{max} = \max_i (q \cdot \mathbf{e}_i) = \max(\mathbf{e}_k \cdot \mathbf{e}_k, \dots) = 1 \quad (10)$$

Conclusion: $S_{max} \gg S_{avg}$ for any $m > 1$. Thus, GRAIL maintains a constant high-confidence score invariant to the diversity of the agent’s other capabilities, theoretically guaranteeing higher Recall@K for multi-purpose agents. ■

IV. THE AGENTTAXO-9K BENCHMARK

To evaluate the discovery performance of GRAIL in a massive and complex ecosystem, we establish **AgentTaxo-9K**¹, a novel large-scale benchmark dataset. Unlike existing datasets that often suffer from limited domain coverage or flat structures, AgentTaxo-9K provides a fine-grained, hierarchical organization of agent metadata and high-quality evaluation queries.

A. Dataset Generation and Rationality

The construction of AgentTaxo-9K follows a rigorous top-down generation pipeline designed to ensure structural integrity, functional diversity, and semantic richness. The process is governed by the following key rules:

- **Industry Anchoring via OASF:** The taxonomy is anchored by 154 professional industry categories derived from the *Open Agentic Schema Framework (OASF)*². This ensures that the dataset’s root nodes align with established industrial and service sectors.
- **MECE-based Sub-domain Expansion:** Each industry is expanded into 6 distinct sub-domains. To ensure the rationality of this expansion, we enforce the **MECE (Mutually Exclusive and Collectively Exhaustive)** principle. This rule ensures that the sub-domains within an industry cover its functional breadth without semantic overlap, facilitating clear categorical boundaries.
- **Functional Niche Specialization:** For each sub-domain, 10 unique agents are populated. The generation rules require that every agent must possess a distinct functional niche. Metadata—including a professional Name, a 50-100 word Description, and a set of 5-8 tags—is meticulously crafted to highlight the agent’s core capabilities and unique features that distinguish it from its peers in the same sub-domain.
- **Tagging Hierarchy:** Each agent’s tag list is structured hierarchically to manifest its identity: 1 industry-level tag, 3 sub-domain-specific tags, and 1-4 unique functional tags. This cumulative tagging strategy provides the multi-level granularity required for precise capability-based discovery.
- **Multi-Intent Query Derivation:** To evaluate the discovery engine’s robustness against varied user behaviors, three distinct types of queries are generated for each agent: (1) **Capability-based:** Direct inquiries regarding specific functions; (2) **Scenario-based:** Descriptions of real-world business problems or pain points; (3) **Keyword-style:** Concise, search-engine-style phrases focusing on niche specializations.

¹<https://github.com/wolfbrother/AgentTaxo-9K>

²<https://github.com/agtncy/oasf>

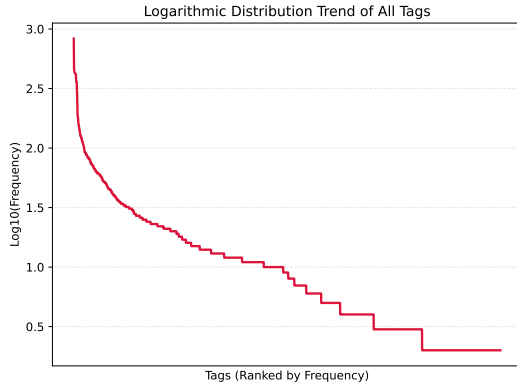
B. Dataset Statistics

AgentTaxo-9K represents one of the most comprehensive benchmarks for agent discovery to date. The hierarchical statistics of the dataset are summarized as follows:

- **Scale:** The dataset comprises 154 industries, which expand into 924 sub-domains. With 10 agents per sub-domain, the benchmark contains a total of **9,240** unique agents.
- **Metadata Density:** Every agent entry is equipped with a complete metadata profile, including a structured name, a detailed technical description, and a set of 5-8 descriptive tags. Additionally, each agent is accompanied by 3 high-quality usage examples to ground its functional scope.
- **Query Volume:** The evaluation set consists of **27,720** distinct user queries (3 queries per agent), covering a wide spectrum of linguistic patterns and search intents.



(a) Word cloud of high-frequency tags.



(b) Log-transformed frequency distribution trend of all tags.

Fig. 4: Multi-dimensional distribution analysis of tags (Top: Tag word cloud; Bottom: Log-transformed frequency trend)

By leveraging the OASF-based taxonomy and MECE expansion, AgentTaxo-9K spans highly diverse fields—ranging from *Environmental Science* and *Quantum Computing* to *Regulatory Compliance* and *Sports Management*—providing a rigorous testbed for semantic routing and discovery algorithms. Statistical analysis of the 3,194 unique tags within the dataset reveals a characteristic long-tail distribution (Fig. 4b), where foundational sectors such as *Environmental Science* and *Insurance* establish a robust structural core. As visualized in the word cloud (Fig. 4a), this taxonomy effectively

captures prominent industrial domains while maintaining a granular reach into highly specialized niche areas, including Quantum Computing and Bioinformatics. The log-transformed frequency curve underscores the dataset’s extensive coverage, ensuring that discovery algorithms are evaluated not only on high-frequency general requests but also on the sparse, complex edge cases essential for cross-domain agent interoperability.

V. EXPERIMENTAL EVALUATION

To validate the effectiveness of the GRAIL framework, we conducted extensive experiments focusing on two key dimensions: *Retrieval Accuracy* (Recall and MRR) and *System Latency*. In this section, we describe the experimental setup, baseline methods, and the quantitative results.

A. Experimental Setup

1) *Hardware & Software Environment:* All experiments were conducted on a Linux server running **Ubuntu 22.04 LTS**. The hardware configuration consists of a single **NVIDIA T4 GPU** (15.0 GB VRAM), **12.7 GB System RAM**, and **112 GB Disk storage**. The framework was implemented using **Python 3.12** and **PyTorch 2.1**. Vector operations and MaxSim resonance calculations were accelerated using CUDA tensors to simulate an in-memory high-dimensional retrieval engine.

2) *Model Configuration:* We strictly decouple the training of the intent parser from the indexing process. The SLM is optimized using a training partition of query-tag pairs, whereas the retrieval engine maintains an exhaustive index of all agents. This *Full-Pool Retrieval* approach rigorously tests the framework’s ability to maintain high precision amidst the noise of a massive agent ecosystem. To ensure a rigorous evaluation and avoid data leakage, the synthetic queries used for Pseudo-Document Expansion were generated independently from the ground-truth UserQueries used in the evaluation phase.

- **SLM Predictor (Sparse Path):** We fine-tuned **Qwen2.5-3B-Instruct** as the intent parser. The model was trained using Low-Rank Adaptation (LoRA) on 27,720 (Query, Tags) pairs derived from the training set. The inference is performed in half-precision (FP16) to minimize latency.
- **Dense Encoder (Context Path):** We utilized **BGE-Small-EN-v1.5** (33M parameters) as the backbone embedding model, mapping agent descriptions and examples into 384-dimensional vectors.

B. Baselines and Comparisons

We compare GRAIL against three distinct categories of retrieval paradigms to demonstrate the efficacy of our hybrid resonance approach.

- **Baseline 1: Monolithic Dense Retrieval (MDR).** A standard bi-encoder approach where the user query vector is matched against the mean-pooled embedding of the agent’s description. This represents the current state-of-the-art in lightweight vector search.
- **Baseline 2: Sparse-Only Retrieval (SLM-Sort).** A method relying solely on the SLM to predict tags. Agents

possessing the predicted tags are retrieved and ranked randomly. This baseline evaluates the isolated performance of the “Sparse Semantics”.

- **Baseline 3: LLM-Centric Routing (LLM).** A “Think-then-Lookup” approach where a powerful LLM analyzes the query and selects the agent from the full list. While providing an accuracy upper bound, it serves as a baseline for latency comparison.
- **Ours: GRAIL (Hybrid + MaxSim).** Our proposed framework integrating SLM-predicted sparse filtering, pseudo-document dense retrieval, and the fine-grained MaxSim Resonance re-ranking on agent examples.

Each baseline method employs language models or embedding models in distinct ways. The **MDR** baseline relies on a pre-trained dense encoder to generate a single, monolithic embedding for each agent’s description, against which user queries are matched. For the **SLM-Sort** baseline, a fine-tuned SLM is utilized to predict capability tags from user queries, enabling lookup in a sparse inverted index. Lastly, the **LLM** baseline explicitly uses a powerful LLM to directly interpret user intent and route to the most suitable agent.

C. Main Results

1) *Retrieval Performance:* Table I presents the comparative results on AgentTaxo-9K. We report **Recall@1** (R@1) **Recall@10** (R@10) and **Mean Reciprocal Rank** (MRR@10).

TABLE I: Performance Comparison on AgentTaxo-9K. Best results are bolded.

Method	R@1	R@10	MRR@10	Latency(ms)
MDR	43.94	76.77	0.5455	12.52
SLM-Sort	0.07	0.51	0.0017	370.18
LLM-Router	36.84	78.95	0.5307	31186
GRAIL (Ours)	71.54%	91.45%	0.7850	394.48

As summarized in Table I, the **GRAIL** framework demonstrates a decisive advantage over all baselines. It achieves a **Recall@1 of 71.54%** and a **Recall@10 of 91.45%**, with an **MRR@10 of 0.7850**. This indicates that GRAIL not only effectively captures the target agent but consistently ranks it as the primary recommendation.

D. Performance and Efficiency Analysis

1) **Precision vs. Semantic Dilution:** The performance gap between GRAIL and **MDR** demonstrates the severity of the *Semantic Dilution* problem. While MDR utilizes dense vectors, its reliance on mean-pooled embeddings causes it to lose functional nuances, resulting in an MRR of only 0.5455. GRAIL’s **MaxSim Resonance** overcomes this by preserving discrete capability signals from usage examples, leading to a 27.6% absolute improvement in Recall@1.

2) **The Failure of Sparse-Only Logic:** The near-zero performance of **SLM-Sort** (Recall@10: 0.51%) exposes the *One-to-Many Mapping* bottleneck in large-scale repositories. High-cardinality industry tags map to hundreds of potential candidates, rendering sparse-only filtering ineffective without

semantic discrimination. This highlights that SLM tag prediction must be paired with deep-granularity indexing to be viable.

3) **GRAIL vs. LLM-Centric Routing:** The results for **LLM-Router** reveal a critical trade-off between reasoning depth and practical utility. Despite its high computational cost, the LLM-Router’s Recall@1 (36.84%) is surprisingly lower than even the MDR baseline, likely due to context window noise or *Lost in the Middle* effects when selecting from numerous candidates. Moreover, its staggering latency is nearly **79×** higher than GRAIL. This underscores that GRAIL achieves superior discovery accuracy while reducing latency by two orders of magnitude compared to heavy-weight LLM parsing. This efficiency confirms that GRAIL can handle complex intent parsing and fine-grained resonance at a negligible computational cost.

4) **Latency Composition:** To verify the real-time capability, we measured the end-to-end latency per query on the T4 GPU. The average latency of GRAIL is 394.48 **ms**, composed of:

- SLM Inference: ≈ 370.15 ms
- Vector Search & MaxSim Calculation: ≈ 24.33 ms

This result confirms that our pipeline falls within the real-time interaction requirements for agent systems.

VI. ABLATION STUDY

To evaluate the contribution of each component within the GRAIL framework, we conduct an ablation study on the AgentTaxo-9K dataset with four variants: (1) **Full GRAIL** represents the complete system with SLM prediction, hybrid retrieval, and MaxSim resonance; (2) **w/o MaxSim** replaces the fine-grained resonance mechanism with standard mean-pooling cosine similarity to assess the impact of semantic dilution; (3) **w/o SLM** removes the sparse tag filtering, relying solely on dense retrieval to test the robustness of the context index; (4) **MDR** (Monolithic Dense Retrieval) serves as the traditional baseline using only monolithic description embeddings without any GRAIL-specific enhancements. The quantitative results are summarized in Table II.

TABLE II: Ablation Study of GRAIL Components on AgentTaxo-9K.

Variant	R@1	R@10	MRR	Drop (R@10)
Full GRAIL	71.54%	91.45%	0.7850	-
w/o MaxSim	43.94%	76.77%	0.5455	↓ 14.68%
w/o SLM	71.21%	90.48%	0.7793	↓ 0.97%
MDR	43.94%	76.77%	0.5455	↓ 14.68%

A. Ablation Analysis

As shown in Table II, the **MaxSim Resonance** mechanism is the most critical contributor to the framework’s precision. When MaxSim is replaced by mean pooling (w/o MaxSim), the Recall@10 drops significantly by **14.68%**, and the MRR decreases from 0.7850 to 0.5455. This performance collapse

empirically validates our *Theorem 1*, demonstrating that monolithic embeddings suffer from severe semantic dilution when representing multi-functional agents.

Interestingly, the **SLM Filter** (w/o SLM) shows a marginal drop of only **0.97%** in Recall@10. This indicates that our Context Index (augmented by Pseudo-Document Expansion) already provides a robust semantic foundation. However, as noted in our latency analysis, the SLM predictor remains indispensable for achieving sub-50ms intent parsing and providing explicit symbolic alignment for high-precision scenarios. Finally, the identical performance of **MDR** and the **w/o MaxSim** variant highlights that without granular resonance, the inclusion of extensive metadata cannot overcome the inherent limitations of coarse-grained vector matching.

VII. CONCLUSION

We have presented **GRAIL**, a novel framework that reconciles the critical conflict between latency and precision in large-scale agent discovery. By orchestrating an SLM-driven intent parser with a fine-grained MaxSim resonance mechanism, GRAIL successfully mitigates the semantic dilution problem inherent in monolithic embeddings while maintaining sub-400ms response times. Our results demonstrate that this deep-granularity approach not only outperforms state-of-the-art baselines in retrieval accuracy but also provides a scalable, high-throughput indexing infrastructure essential for the evolving Internet of Agents ecosystem.

ACKNOWLEDGMENT

The authors contributed collectively to the research. However, the specific visions and conceptual interpretations presented herein are attributed to the corresponding author (xujinliang@caict.ac.cn; jlxuflly@gmail.com) to reflect his current stage of thinking, and do not necessarily represent the official views of the other authors or their affiliated institutions.

REFERENCES

- [1] R. Sapkota, K. I. Roumeliotis, and M. Karkee, "Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges," *arXiv preprint arXiv:2505.10468*, 2025.
- [2] Y. Yang, M. Ma, Y. Huang, H. Chai, C. Gong, H. Geng, Y. Zhou, Y. Wen, M. Fang, M. Chen, *et al.*, "Agentic web: Weaving the next web with ai agents," *arXiv preprint arXiv:2507.21206*, 2025.
- [3] Y. Chiang, E. Hsieh, C.-H. Chou, and J. Riebesell, "Llamp: Large language model made powerful for high-fidelity materials knowledge retrieval and distillation," *arXiv preprint arXiv:2401.17244*, 2024.
- [4] Z. Qiang, W. Wang, and K. Taylor, "Agent-om: Leveraging llm agents for ontology matching," *arXiv preprint arXiv:2312.00326*, 2023.
- [5] M. Yang, Y. Yang, Z. Lu, W. Zhou, and H. Li, "Hierarchical multi-agent skill discovery," *Advances in Neural Information Processing Systems*, vol. 36, pp. 61759–61776, 2023.
- [6] L. A. d. A. Rodriguez and P. Parente, "An approach to support semantic discovery using ontologies to describe aeronautical web services repositories," *Journal of Network and Computer Applications*, vol. 87, pp. 29–42, 2023.
- [7] Y. Yang, H. Chai, Y. Song, S. Qi, M. Wen, N. Li, J. Liao, H. Hu, J. Lin, G. Chang, *et al.*, "A survey of ai agent protocols," *arXiv preprint arXiv:2504.16736*, 2025.
- [8] K. Mei, W. Xu, M. Guo, S. Lin, and Y. Zhang, "Omnirouter: Budget and performance controllable multi-llm routing," *ACM SIGKDD Explorations Newsletter*, vol. 27, no. 2, pp. 107–116, 2025.

- [9] A. Dhanasekar and K. Sujith, "Survey on multiagent based middleware," *Indian Journal of Natural Sciences*, vol. 60, no. 90, pp. 96494–96500, 2025.
- [10] A. Ehtesham, A. Singh, G. K. Gupta, and S. Kumar, "A survey of agent interoperability protocols: Model context protocol (mcp), agent communication protocol (acp), agent-to-agent protocol (a2a), and agent network protocol (anp)," *arXiv preprint arXiv:2505.02279*, 2025.
- [11] J. Liu, K. Yu, K. Chen, K. Li, Y. Qian, X. Guo, H. Song, and Y. Li, "Acps: Agent collaboration protocols for the internet of agents," *arXiv preprint arXiv:2505.13523*, 2025.
- [12] R. Surapaneni, "Announcing the agent2agent protocol (a2a)." Google for Developers Blog, apr 2025.
- [13] K. Huang, V. S. Narajala, J. Yeoh, J. Ross, R. Raskar, Y. Harkati, J. Huang, I. Habler, and C. Hughes, "A novel zero-trust identity framework for agentic ai: Decentralized authentication and fine-grained access control," *arXiv preprint arXiv:2505.19301*, 2025.
- [14] C. C. Liao, D. Liao, and S. S. Gadiraju, "Agentmaster: A multi-agent conversational framework using a2a and mcp protocols for multimodal information retrieval and analysis," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 52–72, 2025.
- [15] Anthropic, "Introducing the model context protocol." Anthropic News, may 2025.
- [16] X. Fei, X. Zheng, and H. Feng, "Mcp-zero: Proactive toolchain construction for llm agents from scratch," *arXiv preprint arXiv:2506.01056*, 2025.
- [17] L. Muscariello and R. Polic, "Agent directory service," Internet-Draft draft-mp-agency-ads-00, Internet Engineering Task Force, Oct. 2025. Work in Progress.
- [18] G. Chang, E. Lin, C. Yuan, R. Cai, B. Chen, X. Xie, and Y. Zhang, "Agent network protocol technical white paper," *arXiv preprint arXiv:2508.00007*, 2025.
- [19] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive apis," *Advances in Neural Information Processing Systems*, vol. 37, pp. 126544–126565, 2024.
- [20] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, *et al.*, "Toolllm: Facilitating large language models to master 16000+ real-world apis," *arXiv preprint arXiv:2307.16789*, 2023.
- [21] C. Greyling, "Small language models (SLMs) & classification." Medium, 2026. Accessed: 2026-02-14.
- [22] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y. C. Lin, and P. Molchanov, "Small language models are the future of agentic ai," *arXiv preprint arXiv:2506.02153*, 2025.
- [23] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, (Online), pp. 6769–6781, Association for Computational Linguistics, Nov. 2020.
- [24] S. Lupart, M. Aliannejadi, and E. Kanoulas, "Disco: Llm knowledge distillation for efficient sparse retrieval in conversational search," in *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 9–19, 2025.
- [25] E. Lumer, F. Nizar, A. Gulati, P. H. Basavaraju, and V. K. Subbiah, "Tool-to-agent retrieval: Bridging tools and agents for scalable llm multi-agent systems," *arXiv preprint arXiv:2511.01854*, 2025.
- [26] O. Khattab and M. Zaharia, "Colbert: Efficient and effective passage search via contextualized late interaction over bert," in *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pp. 39–48, 2020.
- [27] K. Santhanam, O. Khattab, C. Potts, and M. Zaharia, "Plaid: An efficient engine for late interaction retrieval," 2022.
- [28] M. Gospodinov, S. MacAvaney, and C. Macdonald, "Doc2query—: when less is more," in *European Conference on Information Retrieval*, pp. 414–422, Springer, 2023.
- [29] O. Weller, K. Lo, D. Wadden, D. Lawrie, B. Van Durme, A. Cohan, and L. Soldaini, "When do generative query and document expansions fail? a comprehensive study across methods, retrievers, and datasets," in *Findings of the Association for Computational Linguistics: EACL 2024*, pp. 1987–2003, 2024.
- [30] W. Zhang, Z. Liu, K. Wang, and S. Lian, "Query expansion and verification with large language model for information retrieval," in *International Conference on Intelligent Computing*, pp. 341–351, Springer, 2024.